# nesterov-lean: Formal Proofs of Nesterov Accelerated Gradient Descent in Lean 4

Ben Cassie
2026

## Abstract

nesterov-lean is a Lean 4 / Mathlib library formalising the classical O(1/k^2) convergence proof for Nesterov accelerated gradient descent on smooth convex objectives. The library works over a real Hilbert space E, with an objective f : E -> ℝ, a gradient oracle grad_f : E -> E, an L-smoothness hypothesis, and a first-order convexity hypothesis. It defines the accelerated gradient descent iterates, the momentum coefficient β_k = (k - 1)/(k + 2), the auxiliary sequence v_k, and a Lyapunov potential whose non-increase yields the optimal first-order convergence rate. The development contains zero sorry, zero admit, and no project-specific axioms. Its significance is twofold: it provides a machine-checked reference for one of the central algorithms of convex optimisation, and it supplies an importable formal artifact for future work on optimisation, machine learning, acceleration, and AI safety.

## 1. Introduction

Nesterov accelerated gradient descent is one of the foundational algorithms of modern convex optimisation. Introduced by Nesterov in 1983, it improves the classical O(1/k) convergence rate of plain gradient descent to the optimal O(1/k^2) rate for first-order methods on smooth convex objectives. This improvement is not a small constant-factor refinement. It is a qualitative change in the asymptotic behaviour of the method, achieved by adding a carefully tuned momentum term.

The algorithm is now standard in optimisation theory, numerical analysis, machine learning, and control. It is also the conceptual ancestor of many accelerated and momentum-based methods used in large-scale training systems. Arguments about acceleration often rely on a small set of delicate algebraic facts: the momentum coefficient must stay in the correct range, the accelerated iterates must satisfy a hidden mixing identity, a descent inequality must be combined with convex interpolation, and a Lyapunov potential must be chosen so that inner-product and gradient-norm terms cancel exactly.

These facts are usually presented in textbook notation. In informal mathematics, it is easy to write “after rearranging” or “by cancellation” at the most important step. In a proof assistant, every coefficient and side condition has to be made explicit. The rate O(1/k^2) depends on the particular coupling between the momentum coefficient, the auxiliary sequence, and the Lyapunov weights. A wrong factor of two breaks the proof.

nesterov-lean formalises this convergence spine in Lean 4 / Mathlib. It defines the accelerated gradient descent sequence, proves basic bounds on the momentum coefficient, proves the descent lemma for a 1/L gradient step, establishes the key in-

terpolation and norm-expansion identities, proves Lyapunov non-increase, and derives the final `O(1/k^2)` convergence rate.

The contribution is not a new acceleration theorem. It is a machine-checked, importable proof artifact for a classical result. This matters because accelerated methods are basic components in modern machine learning and optimisation. If future formal work is to reason about training dynamics, stability, alignment interventions, or optimality of first-order algorithms, it needs reliable formal foundations for both ordinary gradient descent and accelerated gradient descent.

## 2. Library Overview

The project is organised into three Lean modules plus a root import file:

- `NesterovLean/Defs.lean` defines `LSmooth`, `ConvexFirstOrder`, `IsGlobalMin`, `momentumCoeff`, `agdState`, `agd_x`, `agd_y`, `theta`, `v_seq`, and `lyapunov`.
- `NesterovLean/MomentumStep.lean` proves bounds on `β_k` and the one-step descent lemma.
- `NesterovLean/Convergence.lean` proves Lyapunov non-increase and the `O(1/k^2)` convergence rate.
- `NesterovLean.lean` is the root module importing the library.

The project depends on:

- Lean `v4.28.0`
- Mathlib `v4.28.0`

The formal setting is a real Hilbert space:

```
variable {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
```

The objective is a function `f : E -> ℝ`, equipped with a gradient map `grad_f : E -> E`. Smoothness is represented directly by the quadratic upper-bound inequality:

```
def LSmooth (f : E -> ℝ) (grad_f : E -> E) (L : ℝ) : Prop :=
  ∀ x y : E, f y ≤ f x + ⟪grad_f x, y - x⟫ + L / 2 * ‖y - x‖ ^ 2
```

Convexity is represented by the first-order condition:

```
def ConvexFirstOrder (f : E -> ℝ) (grad_f : E -> E) : Prop :=
  ∀ x y : E, f y ≥ f x + ⟪grad_f x, y - x⟫
```

The accelerated method uses step size `α = 1/L` and momentum coefficient:

```
β_k = (k - 1) / (k + 2).
```

The AGD iterates are:

```
y_{k+1} = x_k - (1/L) • ∇f(x_k),
x_{k+1} = y_{k+1} + β_k • (y_{k+1} - y_k).
```

The proof also introduces the auxiliary sequence:

```
v_0 = x_0,
v_{k+1} = v_k - ((k+1)/(2L)) • ∇f(x_k).
```

The Lyapunov potential is:

`Ψ_k = (k^2 / 4)(f(y_k) - f*) + (L / 2) ‖v_k - x*‖^2.`

The convergence proof shows that `Ψ_{k+1} ≤ Ψ_k`, then extracts the function-value rate from the first term of the potential.

The repository is available at:

https://github.com/velvetmonkey/nesterov-lean

## 3. Theorem Inventory

The library contains ten central theorem and lemma declarations for the accelerated-gradient proof. They divide naturally into momentum properties and convergence machinery.

### Layer 1 - Momentum Properties

1. `momentumCoeff_nonneg` — The momentum coefficient is non-negative for `k ≥ 1`:

```
theorem momentumCoeff_nonneg {k : ℕ} (hk : 1 ≤ k) :
    0 ≤ momentumCoeff k
```

This proves the lower side of the admissible range for `β_k`.

2. `momentumCoeff_lt_one` — The momentum coefficient is strictly less than one:

```
theorem momentumCoeff_lt_one (k : ℕ) :
    momentumCoeff k < 1
```

This proves the upper side of the admissible range and rules out overrelaxation beyond coefficient one.

3. `momentumCoeff_mem_Ico` — For `k ≥ 1`, the momentum coefficient lies in `[0, 1)`:

```
theorem momentumCoeff_mem_Ico {k : ℕ} (hk : 1 ≤ k) :
    momentumCoeff k ∈ Set.Ico (0 : ℝ) 1
```

This packages the preceding two bounds in the interval form used by later reasoning.

4. `descent_lemma` — A gradient step of size `1/L` decreases the function value by at least `(1/(2L)) ‖∇f(x_k)‖^2`:

```
theorem descent_lemma
    (f : E -> ℝ) (grad_f : E -> E) (L : ℝ) (x₀ : E)
    (hL : 0 < L)
    (hSmooth : LSmooth f grad_f L)
    (k : ℕ) :
    f (agd_y grad_f L x₀ (k + 1)) ≤
      f (agd_x grad_f L x₀ k) -
        1 / (2 * L) * ‖grad_f (agd_x grad_f L x₀ k)‖ ^ 2
```

This is the standard smoothness descent inequality specialised to the AGD gradient step `y_{k+1}`.

**Layer 2 - Convergence**

5. `x_eq_combo` — The accelerated iterate is a convex-style mixture of `y_k` and `v_k`:

```
lemma x_eq_combo
    (grad_f : E -> E) (L : ℝ) (x₀ : E) (hL : 0 < L) (k : ℕ) :
    agd_x grad_f L x₀ k =
      (1 - theta k) • agd_y grad_f L x₀ k + theta k • v_seq grad_f L x₀ k
```

This is the hidden structural identity that connects the implemented AGD recurrence to the Lyapunov proof.

6. `convex_interp_bound` — Convexity bounds the function gap at `x_k` by a combination of the gap at `y_k` and an inner-product term:

```
lemma convex_interp_bound
    (f : E -> ℝ) (grad_f : E -> E) (L : ℝ) (x₀ x_star : E)
    (hL : 0 < L)
    (hConvex : ConvexFirstOrder f grad_f)
    (hMin : IsGlobalMin f x_star)
    (k : ℕ) :
    f (agd_x grad_f L x₀ k) - f x_star ≤
      (1 - theta k) * (f (agd_y grad_f L x₀ k) - f x_star) +
        theta k * ⟪grad_f (agd_x grad_f L x₀ k),
          v_seq grad_f L x₀ k - x_star⟫
```

This is the convex interpolation step that brings the auxiliary sequence into the function-value estimate.

7. `key_inequality` — The descent lemma and convex interpolation combine into the per-step inequality:

```
lemma key_inequality
    (f : E -> ℝ) (grad_f : E -> E) (L : ℝ) (x₀ x_star : E)
    (hL : 0 < L)
    (hSmooth : LSmooth f grad_f L)
    (hConvex : ConvexFirstOrder f grad_f)
    (hMin : IsGlobalMin f x_star)
    (k : ℕ) :
    f (agd_y grad_f L x₀ (k + 1)) - f x_star ≤
      (1 - theta k) * (f (agd_y grad_f L x₀ k) - f x_star) +
        theta k * ⟪grad_f (agd_x grad_f L x₀ k),
          v_seq grad_f L x₀ k - x_star⟫ -
        1 / (2 * L) * ‖grad_f (agd_x grad_f L x₀ k)‖ ^ 2
```

The proof is intentionally short: once `descent_lemma` and `convex_interp_bound` are in place, the result follows by linear arithmetic.

8. `v_norm_sq_expand` — The squared distance of the auxiliary sequence expands exactly:

```
lemma v_norm_sq_expand
    (grad_f : E -> E) (L : ℝ) (x₀ x_star : E)
```

```
    (hL : 0 < L)
    (k : ℕ) :
    L / 2 * ‖v_seq grad_f L x₀ (k + 1) - x_star‖ ^ 2 =
      L / 2 * ‖v_seq grad_f L x₀ k - x_star‖ ^ 2 -
        ((k : ℝ) + 1) / 2 * ⟪grad_f (agd_x grad_f L x₀ k),
          v_seq grad_f L x₀ k - x_star⟫ +
        ((k : ℝ) + 1) ^ 2 / (8 * L) *
          ‖grad_f (agd_x grad_f L x₀ k)‖ ^ 2
```

This is the norm-squared expansion that supplies the terms needed to cancel against the per-step inequality.

9. `lyapunov_nonincrease` — The Lyapunov potential is non-increasing:

```
theorem lyapunov_nonincrease
    (f : E -> ℝ) (grad_f : E -> E) (L : ℝ) (x₀ x_star : E)
    (hL : 0 < L)
    (hSmooth : LSmooth f grad_f L)
    (hConvex : ConvexFirstOrder f grad_f)
    (hMin : IsGlobalMin f x_star)
    (k : ℕ) :
    lyapunov f grad_f L x₀ x_star (k + 1) ≤
      lyapunov f grad_f L x₀ x_star k
```

This is the central Lyapunov theorem of the library.

10. `convergence_rate` — Nesterov accelerated gradient descent achieves the `O(1/k^2)` rate:

```
theorem convergence_rate
    (f : E -> ℝ) (grad_f : E -> E) (L : ℝ) (x₀ x_star : E)
    (hL : 0 < L)
    (hSmooth : LSmooth f grad_f L)
    (hConvex : ConvexFirstOrder f grad_f)
    (hMin : IsGlobalMin f x_star)
    {k : ℕ} (hk : 1 ≤ k) :
    f (agd_y grad_f L x₀ k) - f x_star ≤
      2 * L * ‖x₀ - x_star‖ ^ 2 / k ^ 2
```

This is the headline theorem: for every `k ≥ 1`, the function-value error at `y_k` is bounded by `2L ‖x₀ - x*‖^2 / k^2`.

## 4. Key Technical Highlights

### The Mixing Identity

The theorem `x_eq_combo` states:

```
x_k = (1 - θ_k) y_k + θ_k v_k,
where θ_k = 2 / (k + 1).
```

This identity is not an arbitrary rewriting trick. It is the algebraic bridge between the

explicit AGD update and the Lyapunov proof. The implemented algorithm updates `x_{k+1}` from `y_{k+1}` and `y_k` using the momentum coefficient `β_k = (k - 1)/(k + 2)`. The convergence proof, however, is most naturally expressed using the auxiliary sequence `v_k` and the coupling parameter `θ_k`.

The choice `θ_k = 2/(k+1)` is therefore structural. It is tuned so that the recurrence for `v_k`, the recurrence for `x_k`, and the momentum coefficient all describe the same accelerated trajectory. If the coefficient were changed independently, the mixture identity would fail and the later cancellation in the Lyapunov proof would no longer line up.

In Lean, `x_eq_combo` is proved by induction over `k`. The base case reduces to the initial condition `x_0 = y_0 = v_0`. The successor case unfolds `agd_x_succ`, `agd_y_succ`, `v_seq_succ`, `momentumCoeff`, and `theta`, then closes the coefficient algebra.

### The Key Inequality

The lemma `key_inequality` combines two ingredients. First, `descent_lemma` gives:

```
f(y_{k+1}) ≤ f(x_k) - (1/(2L)) ‖∇f(x_k)‖^2.
```

Second, `convex_interp_bound` uses the mixing identity and first-order convexity to bound:

```
f(x_k) - f* ≤
  (1 - θ_k)(f(y_k) - f*) +
  θ_k ⟪∇f(x_k), v_k - x*⟫.
```

Substituting the second inequality into the first yields:

```
f(y_{k+1}) - f* ≤
  (1 - θ_k)(f(y_k) - f*) +
  θ_k ⟪∇f(x_k), v_k - x*⟫ -
  (1/(2L)) ‖∇f(x_k)‖^2.
```

The Lean proof reflects the mathematical simplicity of the step. Once the two prior lemmas are established, `key_inequality` follows by linear arithmetic. This is a useful pattern in formal optimisation: the difficult work is often isolating the right intermediate inequalities so that the final combination is routine.

### Lyapunov Cancellation

The Lyapunov potential is:

```
Ψ_k = (k^2 / 4)(f(y_k) - f*) + (L / 2) ‖v_k - x*‖^2.
```

The theorem `lyapunov_nonincrease` proves `Ψ_{k+1} ≤ Ψ_k`. This is where Nesterov acceleration becomes a precise algebraic cancellation rather than a heuristic momentum story.

The proof combines `key_inequality` with `v_norm_sq_expand`. The key inequality contributes an inner-product term involving `⟪∇f(x_k), v_k - x*⟫` and a negative gradient-

norm term. The norm expansion for `v_{k+1}` contributes the corresponding inner-product and gradient-norm terms with coefficients determined by `(k+1)/(2L)`.

The coefficients in `θ_k`, `v_seq`, and the Lyapunov weight `k^2/4` are chosen so that these terms cancel exactly. What remains is controlled by non-negativity of the function gap `f(y_k) - f*`, which follows from the global-minimiser hypothesis. The result is a clean monotonicity statement:

`Ψ_{k+1} ≤ Ψ_k.`

### The Final Rate

The theorem `convergence_rate` extracts the rate from Lyapunov monotonicity. Since `lyapunov_nonincrease` holds at every step, induction gives:

`Ψ_k ≤ Ψ_0.`

At step zero, the Lyapunov potential reduces to:

`Ψ_0 = (L / 2) ‖x_0 - x*‖^2.`

The first term of `Ψ_k` is:

`(k^2 / 4)(f(y_k) - f*).`

The second term is non-negative. Therefore:

`(k^2 / 4)(f(y_k) - f*) ≤ (L / 2) ‖x_0 - x*‖^2.`

Dividing by `k^2/4` gives:

`f(y_k) - f* ≤ 2L ‖x_0 - x*‖^2 / k^2.`

This is the optimal first-order rate for smooth convex objectives. The formal theorem states the bound for every natural number `k` satisfying `1 ≤ k`.

### Standard Axioms Only

The library introduces no project-specific axioms. It is written against Lean 4 and Mathlib, uses ordinary classical mathematics where needed, and contains zero `sorry` and zero `admit`.

## 5. Relation to Sibling Libraries

`nesterov-lean` is part of the same Lean 4 formalisation programme as `gradient-descent-lean`, `hopfield-lean`, and `kuramoto-lean`.

`gradient-descent-lean` formalises deterministic gradient descent convergence for smooth convex optimisation. Its headline result is the standard `O(1/k)` rate for plain gradient descent, together with a geometric rate under strong convexity. `nesterov-lean` extends this optimisation spine by adding momentum and proving the accelerated `O(1/k^2)` rate.

`hopfield-lean` formalises a discrete Lyapunov argument for Hopfield networks. It proves that asynchronous state updates never increase energy and that an infinite

sequence of non-trivial updates is impossible in the finite state space. `nesterov-lean` uses a continuous Lyapunov potential rather than a finite-state energy argument, but the proof pattern is recognisably similar: define a quantity that cannot increase, then extract convergence from that monotonicity.

`kuramoto-lean` formalises finite-N Kuramoto synchronisation dynamics, including gradient identities and Lyapunov descent for coupled oscillator systems. It shows how synchronisation can be expressed through energy-like structure. `nesterov-lean` sits on the optimisation side of the same mathematical landscape: it uses a carefully engineered potential to prove accelerated convergence.

Together, these libraries cover several recurring proof patterns in learning and dynamical systems:

- plain gradient descent and convex optimisation;
- accelerated first-order optimisation;
- discrete attractor dynamics and energy descent;
- synchronisation and Lyapunov descent in coupled systems.

The shared value is that each proof becomes an importable Lean artifact. Future formal work can build on checked components rather than re-establishing foundational convergence facts from scratch.

## 6. Significance for AI Safety

Accelerated first-order optimisation is central to modern machine learning. Momentum and acceleration appear throughout large-scale optimisation, including training algorithms and their theoretical simplifications. While production optimisers often include stochasticity, adaptivity, clipping, normalisation, and implementation details beyond classical Nesterov AGD, the deterministic smooth-convex result remains a foundational reference point.

The `O(1/k^2)` rate matters because it is optimal for first-order methods on smooth convex objectives. This means Nesterov acceleration is not merely an engineering heuristic; it reaches the best possible worst-case order in the standard oracle model. Formalising this proof gives future AI-safety work a checked base for reasoning about when acceleration is justified, which hypotheses it depends on, and how fragile the rate is to changes in coefficients or update structure.

Safety-relevant arguments often involve claims about training dynamics, stability, monotonic improvement, or convergence to desirable states. Those claims can hide side conditions. In the AGD proof, the side conditions include smoothness, convexity, a positive smoothness constant, exact gradient steps, the precise momentum schedule, the auxiliary sequence, and the Lyapunov weighting. Lean forces all of these assumptions into the theorem statements.

A machine-checked AGD convergence theorem does not certify a deployed AI system by itself. It does, however, provide a reliable component for larger formal arguments about optimisation and learning. It can support future developments involving accelerated algorithms, potential-function proofs, training dynamics, or formal comparisons between optimisation methods.

The broader value is cumulative. `gradient-descent-lean` supplies the plain-gradient baseline. `nesterov-lean` supplies the accelerated rate. `hopfield-lean` supplies a discrete Lyapunov convergence proof. `kuramoto-lean` supplies synchronisation and coupled-dynamics structure. Together they form a small but growing library of checked mathematical mechanisms relevant to learning, memory, stability, and alignment.

## 7. Conclusion

`nesterov-lean` provides a compact Lean 4 / Mathlib formalisation of Nesterov accelerated gradient descent for smooth convex objectives over real Hilbert spaces. It defines the AGD iterates, the momentum coefficient, the auxiliary sequence, and the Lyapunov potential used in the classical proof. It proves momentum bounds, the descent lemma, the mixing identity, convex interpolation, the key per-step inequality, the auxiliary norm expansion, Lyapunov non-increase, and the final `O(1/k^2)` convergence rate.

The project is deliberately focused. It does not formalise stochastic acceleration, adaptive optimisers, nonconvex acceleration, lower bounds for first-order methods, or implementation-level training systems. Instead, it supplies a reliable formal core for the classical deterministic theorem. That core can now be imported and extended by future work on optimisation, machine learning, acceleration, control, and AI safety.

## References


Nesterov, Y. (1983). *A method for unconstrained convex problem with the rate of convergence O(1/k²)*. Doklady AN USSR, 269, 543-547.

The Mathlib Community. (2024). *The Lean Mathematical Library*. GitHub repository. https://github.com/leanprover-community/mathlib4

Cassie, B. (2026). *gradient-descent-lean*. Zenodo. DOI: 10.5281/zenodo.20472996. https://doi.org/10.5281/zenodo.20472996

Cassie, B. (2026). *kuramoto-lean*. Zenodo. DOI: 10.5281/zenodo.20468619. https://doi.org/10.5281/zenodo.20468619

Cassie, B. (2026). *hopfield-lean*. Zenodo. DOI: 10.5281/zenodo.20474169. https://doi.org/10.5281/zenodo.20474169

Cassie, B. (2026). *nesterov-lean: Formal Proofs of Nesterov Accelerated Gradient Descent in Lean 4*. GitHub repository. https://github.com/velvetmonkey/nesterov-lean